

UNIT 3

- 1) Justify the impact of normalization on database? Explain 1st, 2nd normal form, 3rd normal form and BCNF with example.

💡 Impact (Importance) of Normalization on Database

Normalization is the process of organizing data in a database to **reduce redundancy** and **improve data integrity**.

It divides large, complex tables into smaller, related ones and defines relationships among them.

✅ Impact / Advantages:

1. Removes Data Redundancy:

Duplicate data is eliminated; storage is optimized.

→ e.g., Student details stored once instead of repeating in every record.

2. Ensures Data Consistency:

Updates made in one place reflect everywhere.

3. Improves Data Integrity:

Reduces chances of anomalies (insertion, deletion, update).

4. Enhances Query Performance:

Smaller, well-structured tables are easier to maintain and search.

5. Simplifies Maintenance:

Easier to modify structure or add new data without affecting existing data.

6. Avoids Anomalies:

- **Insertion anomaly** – can't insert due to missing data.
- **Deletion anomaly** – deleting one record may delete useful data.
- **Update anomaly** – updating one place but not others causes inconsistency.

Normal Forms

Normalization is carried out in steps called **Normal Forms (NF)**.

1 First Normal Form (1NF)

Rule:

- Each cell must contain **a single atomic value** (no repeating groups or arrays).
- Each record must be unique.

◆ Example (Before 1NF):

StudentID	StudentName	Subjects
1	Ravi	DBMS, SE
2	Priya	CN, DBMS, AI

Problem: "Subjects" column has multiple values — not atomic.

✅ After 1NF:

StudentID	StudentName	Subject
1	Ravi	DBMS
1	Ravi	SE
2	Priya	CN
2	Priya	DBMS
2	Priya	AI



Normal Forms

Normalization is carried out in steps called **Normal Forms (NF)**.

1 First Normal Form (1NF)

Rule:

- Each cell must contain **a single atomic value** (no repeating groups or arrays).
- Each record must be unique.

Example (Before 1NF):

StudentID	StudentName	Subjects
1	Ravi	DBMS, SE
2	Priya	CN, DBMS, AI

Problem: "Subjects" column has multiple values — not atomic.

After 1NF:

StudentID	StudentName	Subject
1	Ravi	DBMS
1	Ravi	SE
2	Priya	CN
2	Priya	DBMS
2	Priya	AI



2 Second Normal Form (2NF)

Rules:

- Must be in **1NF**, and
- **No partial dependency:**
No non-key attribute should depend on a **part of a composite primary key**.

◆ Example (Before 2NF):

StudentID	CourseID	StudentName	CourseName
1	DB01	Ravi	DBMS
2	CN01	Priya	CN

Composite Key: (StudentID, CourseID)

Problem:

- StudentName depends **only on StudentID**
- CourseName depends **only on CourseID**
→ Partial dependencies exist.

✅ After 2NF:

Split into two tables:

Student Table

StudentID	StudentName
1	Ravi
2	Priya

Course Table

CourseID	CourseName
DB01	DBMS
CN01	CN



Enrollment Table

StudentID	CourseID
1	DB01
2	CN01

Now no attribute depends on part of the key → Table is in **2NF**.

3 Third Normal Form (3NF)

Rules:

- Must be in **2NF**, and
- **No transitive dependency:**
Non-key attributes should not depend on other non-key attributes.

◆ Example (Before 3NF):

StudentID	StudentName	DeptID	DeptName
1	Ravi	D1	Computer
2	Priya	D2	IT

Problem:

- DeptName depends on DeptID (non-key → non-key dependency).
→ Transitive dependency exists.

✓ After 3NF:

Student Table

StudentID	StudentName	DeptID
1	Ravi	D1
2	Priya	D2

Department Table

DeptID	DeptName
D1	Computer
D2	IT

Now, every non-key attribute depends **only on the primary key** → Table is in **3NF**.

4 Boyce-Codd Normal Form (BCNF)

Rule:

- For every **functional dependency** ($X \rightarrow Y$), **X must be a super key**.
- A stronger version of 3NF.

◆ Example (Before BCNF):

Course	Instructor	Room
DBMS	Smith	R1
OS	John	R2
DBMS	John	R1

Functional dependencies:

- ($\text{Course} \rightarrow \text{Room}$)
- ($\text{Room} \rightarrow \text{Instructor}$)

Here, **Room** $\rightarrow$ **Instructor** violates BCNF because "Room" is not a super key.

✔ After BCNF:

Table 1: Room–Instructor

Room	Instructor
R1	Smith
R2	John

Table 2: Course–Room

Course	Room
DBMS	R1
OS	R2

Now all determinants are super keys $\rightarrow$ Table is in **BCNF**.

Summary Table

Normal Form	Removes	Key Rule
1NF	Repeating groups	Atomic values
2NF	Partial dependency	No attribute depends on part of composite key
3NF	Transitive dependency	Non-key attribute depends only on key
BCNF	Remaining anomalies	Every determinant is a super key

2) Elaborate the significance of codd's rule. Explain 12 rules proposed by codd's.

🔗 Significance of Codd's Rules

E. F. Codd, the father of the Relational Database Model, proposed 12 rules (actually 13, numbered 0 to 12) in 1985 to define what qualifies as a true relational database management system (RDBMS).

✅ Significance:

1. They define the foundation of relational databases.
2. Help to differentiate relational databases from simple file-based systems.
3. Ensure data independence, integrity, and consistency.
4. Encourage standardization in database design and operations.
5. Provide a guideline for database vendors to develop robust RDBMS software.
6. Improve data security, logical structure, and ease of access.

🔗 Codd's 12 Rules (Rule 0–12)

Let's understand each in simple terms 📌

Rule No.	Rule Name	Explanation
Rule 0:	Foundation Rule	Any system that claims to be an RDBMS must manage data only through relational capabilities — i.e., data must be stored in tables, not files or other formats.
Rule 1:	Information Rule	All information (data, metadata) in the database must be represented in table form , with rows and columns.
Rule 2:	Guaranteed Access Rule	Each piece of data (value) should be accessible using table name, primary key, and column name — no need for physical addresses or pointers.
Rule 3:	Systematic Treatment of Null Values	The system must support NULL values (unknown or inapplicable data) in a consistent way, distinct from zero or empty string.
Rule 4:	Dynamic Online Catalog Based on the Relational Model	The database catalog (data dictionary) should also be stored in the form of tables and accessible using SQL.
Rule 5:	Comprehensive Data Sublanguage Rule	The database must support at least one comprehensive language (like SQL) for data definition, manipulation, and control (DDL, DML, DCL).

Rule 6:	View Updating Rule	All theoretically updatable views (virtual tables) must be updatable through the system.
Rule 7:	High-Level Insert, Update, and Delete Rule	The system must support set-level operations (on multiple rows) for insertion, update, and deletion—not just one record at a time.
Rule 8:	Physical Data Independence	Changes in storage structure or access methods should not affect the logical structure or user operations.
Rule 9:	Logical Data Independence	Changes in logical structure (like adding new fields or tables) should not affect application programs.
Rule 10:	Integrity Independence	Integrity constraints (like primary key, foreign key) must be defined in the database , not in the application program.
Rule 11:	Distribution Independence	Users should be unaware of data distribution — whether it is stored locally or on remote servers (transparent access).
Rule 12:	Nonsubversion Rule	If a system provides low-level access (bypassing SQL) , it must not bypass integrity rules or security constraints.

3) What is the impact of insert, update and delete anomaly on overall design of database? How is normalization used to remove these anomalies?

💡 Impact of Insert, Update, and Delete Anomalies on Database Design

In a **poorly designed (unnormalized)** database, data is often **repeated** and **stored redundantly**. This leads to several problems known as **data anomalies**, which affect **data integrity, accuracy, and consistency**.

◆ 1 Insertion Anomaly

Meaning:

Occurs when certain data **cannot be inserted** into the database **without the presence of other data**.

Example:

StudentID	StudentName	Course
1	Ravi	DBMS

If a new course (e.g., "AI") starts but no student has enrolled yet, we **cannot insert** that course because the table requires student information.

Impact:

- Leads to **incomplete data** storage.
- Prevents addition of valid new entries.

◆ 2 Update Anomaly

Meaning:

Occurs when the same data is **stored in multiple places**, and a change in one place is **not reflected** in others.

Example:

StudentID	StudentName	Course	Instructor
1	Ravi	DBMS	Smith
2	Priya	DBMS	Smith

If instructor "Smith" changes to "John," we must update **both rows**.

If we forget to update one, data becomes inconsistent.

Impact:

- Causes **inconsistency** in data.
- Leads to **errors** and loss of reliability.

◆ 3 Deletion Anomaly

Meaning:

Occurs when deleting some data **accidentally removes useful information**.

Example:

StudentID	StudentName	Course
1	Ravi	DBMS

If we delete Ravi's record (student drops out),
we also **lose information about the course "DBMS"**, even though it still exists.

Impact:

- Results in **unintentional data loss**.
- Affects **data integrity** and **future use**.

🔍 How Normalization Removes These Anomalies

Normalization is the process of organizing data into related tables to **eliminate redundancy** and **avoid anomalies**.

✅ 1NF (First Normal Form)

- Removes repeating groups and ensures atomic values.
- Prevents **insertion anomalies** due to non-atomic fields.

✅ 2NF (Second Normal Form)

- Removes **partial dependency** (attributes depending on part of composite key).
- Ensures data about entities like students, courses, and instructors are stored in **separate tables**.
- Reduces **update anomalies**.

✅ 3NF (Third Normal Form)

- Removes **transitive dependencies** (non-key → non-key).
- Keeps data **logically separated**, avoiding **deletion anomalies**.

✅ BCNF (Boyce–Codd Normal Form)

- Strengthens dependency rules so that **every determinant is a key**, ensuring **maximum consistency**.

📄 Summary Table

Type of Anomaly	Cause	Effect on Design	Normalization Solution	🔍
Insertion	Missing data dependency	Incomplete data	1NF & 2NF	
Update	Data redundancy	Inconsistent data	2NF & 3NF	
Deletion	Data coupling	Loss of useful info	3NF & BCNF	

🔹 Conclusion

Unnormalized tables cause serious **data integrity issues** due to insert, update, and delete anomalies.

By applying **normalization (up to 3NF or BCNF)**, a database becomes **consistent, reliable, and efficient**, improving the **overall design and data management**.

4) Enlist differences between 1st, 2nd, 3rd and, BCNF

❖ Differences between 1NF, 2NF, 3NF, and BCNF

Normal Form	Main Rule / Condition	Removes	Dependency Type Allowed	Example Problem Fixed	Key Point
1NF (First Normal Form)	Each cell should contain atomic (single) values and each record should be unique.	Repeating groups, multivalued attributes	Only simple key dependency	Multiple values in one column (e.g., Subjects = DBMS, SE)	Ensures atomicity
2NF (Second Normal Form)	Must be in 1NF and no partial dependency (non-key attribute should depend on full composite key).	Partial dependency	Fully functional dependency	Attributes depend only on part of composite key	Ensures full dependency
3NF (Third Normal Form)	Must be in 2NF and no transitive dependency (non-key → non-key).	Transitive dependency	Direct key → non-key only	DeptName depends on DeptID, not StudentID	Ensures data depends only on key
BCNF (Boyce-Codd Normal Form)	Must be in 3NF and for every dependency $X \rightarrow Y$, X must be a super key .	Remaining anomalies from 3NF	Only super key dependencies	Room → Instructor type dependency	Stronger version of 3NF

📄 Summary:

- 1NF → Removes repeating data
- 2NF → Removes partial dependency
- 3NF → Removes transitive dependency
- BCNF → Removes remaining anomalies by ensuring every determinant is a key

5) Explain different features of good relational database design.

🔹 Features of a Good Relational Database Design

A **good relational database design** ensures that data is stored efficiently, accurately, and can be easily retrieved and maintained.

It should minimize redundancy and maintain data integrity while supporting the required operations.

✅ 1 Data Integrity and Consistency

- Ensures that data in the database remains **accurate and reliable**.
- Uses **Primary Keys**, **Foreign Keys**, and **Constraints** (like `NOT NULL`, `UNIQUE`, `CHECK`) to maintain correctness.

Example: Preventing duplicate student IDs in a student table.

✅ 2 Data Redundancy Control

- Avoids storing the same data multiple times.
- Reduces chances of **update, insert, and delete anomalies**.
- Achieved through **Normalization** (1NF, 2NF, 3NF, BCNF).

✅ 3 Proper Use of Keys

- **Primary Key** → uniquely identifies each record.
- **Foreign Key** → maintains relationships between tables.
- **Candidate Key** and **Alternate Key** → provide flexibility and uniqueness.

These ensure logical connections between tables.

✅ 4 Normalization

- Organizes data into well-structured tables.
- Eliminates redundancy and ensures **data dependency is logical**.
- Improves **data integrity** and simplifies maintenance.

✅ 5 Referential Integrity

- Maintains valid relationships between tables using **foreign keys**.
- Ensures that references between tables remain consistent (no orphan records).

Example: A student record cannot refer to a non-existing department.

✓ 6 Data Independence

- Changes in **physical storage** or **logical structure** should not affect application programs.
 - Ensures **flexibility and scalability** of the system.
-

✓ 7 Simplicity and Clarity

- Tables should represent **one entity** (e.g., Student, Course, Department).
 - Column names should be **meaningful and self-explanatory**.
 - Simplifies database understanding and reduces design complexity.
-

✓ 8 Efficient Query Performance

- Proper **indexing**, **foreign key relationships**, and **optimized table structures** help in faster data retrieval.
 - Queries should run efficiently even on large datasets.
-

✓ 9 Scalability and Flexibility

- The design should allow for **future growth** — new tables, fields, or relationships can be added without major restructuring.
-

✓ 10 Security and Access Control

- Supports **user authentication**, **role-based permissions**, and **access levels** to protect sensitive data.

6) Explain Codd's rules with suitable examples

🔹 Codd's 12 Rules of Relational Database (with Examples)

E.F. Codd proposed **13 rules (Rule 0 to 12)** to define what a **true Relational Database Management System (RDBMS)** should be.

Each rule ensures **data integrity, independence, and relational consistency**.

Rule 0 – Foundation Rule

A system must use relational capabilities to manage the database.

Meaning:

If a system calls itself an RDBMS, it must **store and manipulate data using relational tables** only.

Example:

MySQL or Oracle uses **tables (rows and columns)** — not flat files or hierarchies.

Rule 1 – Information Rule

All information in a database must be represented logically as values in **tables**.

Example:

Student data is stored in a table:

StudentID	Name	Age
1	Ravi	20

Everything — even metadata — is represented in **rows and columns**.

Rule 2 – Guaranteed Access Rule

Each data item must be accessible by **table name, primary key, and column name**.

Example:

To access Ravi's name:

→ Table: `Student`, Key: `StudentID = 1`, Column: `Name`.

No need to know physical storage location.

Rule 3 – Systematic Treatment of Null Values

Null values must be **supported and treated consistently**.

Example:

If a student's phone number is unknown → store as `NULL`, not 0 or "N/A".

This helps differentiate **unknown** vs **empty** data.

Rule 4 – Dynamic Online Catalog Based on the Relational Model

The database catalog (data dictionary) must also be stored in **table form** and accessible using SQL.

Example:

You can query metadata using:

sql

 Copy code

```
SELECT * FROM INFORMATION_SCHEMA.TABLES;
```

This retrieves all tables in the database.

Rule 5 – Comprehensive Data Sublanguage Rule

The database must support at least **one language (like SQL)** that can perform all operations:

- Data Definition (DDL)
- Data Manipulation (DML)
- Data Control (DCL)
- Transaction Control (TCL)

Example:

SQL provides commands like:

```
CREATE TABLE , SELECT , INSERT , UPDATE , DELETE , COMMIT .
```

Rule 6 – View Updating Rule

All views (virtual tables) that can theoretically be updated must be **updatable** through the system.

Example:

If you create a view:

sql

 Copy code

```
CREATE VIEW StudentNames AS SELECT Name FROM Student;
```

Then updating this view should update the original table as well.

Rule 7 – High-Level Insert, Update, and Delete

The system must support **set-level operations** (not just one record at a time).

Example:

You can update multiple records in one query:

sql

 Copy code

```
UPDATE Student SET Age = Age + 1 WHERE Dept = 'CSE';
```

→ Updates all CSE students at once.

Rule 8 – Physical Data Independence

Changes in **storage structure or access methods** must not affect the logical data model.

Example:

Moving the database to another disk or changing the indexing structure should not affect how users query data.

Rule 9 – Logical Data Independence

Changes in **logical structure** (like adding new columns) should not affect application programs.

Example:

If a new column `Email` is added to the `Student` table, existing queries using `Name` and `Age` should still work.

Rule 10 – Integrity Independence

Integrity constraints (like primary key, foreign key, etc.) should be **defined in the database**, not in application code.

Example:

sql

 Copy code

```
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  DeptID INT,  
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

Integrity rules are stored in the database itself.

Rule 11 – Distribution Independence

A user should be unaware of whether data is stored **locally or distributed** across multiple locations.

Example:

A query like:

sql

 Copy code

```
SELECT * FROM Student;
```

Should give the same result whether data is on one server or multiple remote servers.

Rule 12 – Nonsubversion Rule

If a system provides low-level (record-level) access, it must not bypass integrity or security rules defined by the RDBMS.

Example:

Even if an administrator edits a data file directly,
they cannot violate constraints like primary keys or foreign keys.

7) Explain entity and referential integrity constraints used in SQL.

Entity and Referential Integrity Constraints in SQL

Integrity constraints are rules that ensure the accuracy and consistency of data in a database.

Two important integrity constraints in the Relational Model are:

- Entity Integrity Constraint
- Referential Integrity Constraint

1 Entity Integrity Constraint

Definition:

Entity Integrity ensures that every table (relation) has a unique and identifiable row — that is, Primary Key values must be unique and not NULL.

Explanation:

- Each table must have a Primary Key that uniquely identifies every record.
- The Primary Key column(s) cannot contain NULL values, because NULL represents unknown data, and an unknown value cannot uniquely identify a record.
- This maintains the uniqueness and existence of each entity (row).

Example:

sql

Copy code

```
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Age INT  
);
```

StudentID	Name	Age
1	Ravi	20
2	Priya	21
NULL	Aman	19 ✗ Not allowed
1	Rohan	22 ✗ Duplicate key not allowed

✓ Here, StudentID uniquely identifies each student — enforcing Entity Integrity.

◆ 2 Referential Integrity Constraint

Definition:

Referential Integrity ensures that relationships between tables remain consistent.

A Foreign Key in one table must always refer to a valid Primary Key in another table.

Explanation:

- A Foreign Key establishes a relationship between two tables.
- The foreign key value must either:
 - Match an existing primary key value in the parent table, or
 - Be NULL (if allowed).
- This prevents orphan records — records referring to non-existing entities.

Example:

sql

 Copy code

```
CREATE TABLE Department (  
  DeptID INT PRIMARY KEY,  
  DeptName VARCHAR(50)  
);  
  
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  DeptID INT,  
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

Department Table

DeptID	DeptName
1	Computer
2	IT

Student Table

StudentID	Name
1	Ravi
2	Priya
3	Rohan

✅ Referential Integrity ensures that every student's DeptID exists in the Department table.

📄 Summary Table

Constraint	Ensures	Key Used	NULL Allowed?	Example Violation
Entity Integrity	Each record is uniquely identified	Primary Key	❌ Not allowed	Duplicate or NULL key
Referential Integrity	Valid relationship between tables	Foreign Key	✅ Allowed (if optional)	Foreign key refers to non-existing primary key

💬 Conclusion

- Entity Integrity maintains the **uniqueness** of records within a table.
 - Referential Integrity maintains the **consistency of relationships** between tables.
- Together, they ensure the **reliability, accuracy, and stability** of relational databases.

8) What are relational integrity constraints. Explain with example Domain constraints, Referential-Integrity and enterprise constraints.

🔹 Relational Integrity Constraints

Definition:

Relational Integrity Constraints are **rules** that ensure the **accuracy, consistency, and validity** of data in a relational database.

These constraints prevent invalid data from being entered into tables and maintain relationships among data.

◆ 1. Domain Constraints

Definition:

Domain constraints ensure that the **values in each column (attribute)** come from a **specific domain** — i.e., they must be of a valid data type, range, or set of values.

Example:

sql

📄 Copy code

```
CREATE TABLE Student (  
  RollNo INT,  
  Name VARCHAR(50),  
  Age INT CHECK (Age >= 18 AND Age <= 25)  
);
```

✅ Here,

- `Age` must be between **18 and 25** → domain constraint.
- If you try to insert `Age = 30`, the database will reject it.

◆ 2. Entity Integrity Constraint

Definition:

Entity integrity ensures that every **table (relation)** has a **primary key**, and the **primary key cannot be NULL or duplicate**.

This guarantees that each record can be **uniquely identified**.

Example:

sql

 Copy code

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

✓ Here,

- `RollNo` is the **primary key**.
- You cannot insert a record with `NULL` or duplicate `RollNo`.

◆ 3. Referential Integrity Constraint

Definition:

Referential integrity ensures that **foreign key values** in one table must **match** a **primary key value** in another table — or be **NULL**.

It maintains **valid relationships** between tables.

Example:

sql

 Copy code

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

✅ Here,

- Each `DeptID` in the **Employee** table must exist in the **Department** table.
- If a department is deleted, you can't have employees assigned to that deleted department (unless handled using `ON DELETE` rules).

◆ 4. Enterprise Constraints (Business Rules)

Definition:

Enterprise constraints are **user-defined rules** that enforce specific **business policies or logic** in the organization.

They go beyond standard SQL constraints.

Example:

- A company policy states that **salary cannot exceed ₹1,00,000**.
- Or a student can register for a **maximum of 5 subjects**.

sql

Copy code

```
CREATE TABLE Employee (  
  EmpID INT PRIMARY KEY,  
  EmpName VARCHAR(50),  
  Salary DECIMAL(10,2) CHECK (Salary <= 100000)  
);
```

✓ Here,

The `CHECK` condition enforces the enterprise rule on salary.

📄 Summary Table

Constraint Type	Purpose	Example
Domain Constraint	Validates data type & range	Age BETWEEN 18 AND 25
Entity Integrity	Ensures unique, non-null primary key	PRIMARY KEY (RollNo)
Referential Integrity	Maintains valid foreign key relationships	FOREIGN KEY (DeptID)
Enterprise Constraint	Enforces business rules	CHECK (Salary <= 100000)

- 9) What is functional dependency? Explain its use in database design. [9] Consider the following schema. Student (RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks) Identify the functional dependencies and check whether the given schema is in 3NF or not. If not justify and convert the schema into 3NF.

1. What is a Functional Dependency (FD)?

A functional dependency $X \rightarrow Y$ between attribute sets in a relation R means: **for any two tuples in R, if they agree on X, they must agree on Y.**

In other words, the value of X **functionally determines** the value of Y.

Example: $\text{RollNo} \rightarrow \text{Branch_code}$ means a given RollNo always has exactly one Branch_code.

2. Uses of Functional Dependencies in database design

1. **Identify candidate keys** — FDs help find minimal attribute sets that determine all other attributes.
2. **Normalize relations** — FDs reveal redundancies (partial & transitive dependencies) and guide decomposition into 1NF, 2NF, 3NF, BCNF.
3. **Prevent anomalies** — by decomposing on FDs you reduce insert/update/delete anomalies.
4. **Enforce integrity & constraints** — FDs correspond to keys and constraints implemented in SQL (PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK).
5. **Query optimization & schema design** — knowing FDs enables better indexing and query planning.

3. Given schema

Student(RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks)

a) Identify likely functional dependencies

From typical semantics we can infer:

1. $\text{RollNo} \rightarrow \text{Branch_code}$
(a student has one branch)
2. $(\text{RollNo}, \text{Exam_Name}) \rightarrow \text{Marks_Obtained}$
(the marks a student obtained in a particular exam)
3. $\text{Exam_Name} \rightarrow \text{Total_Marks}$
(each exam has a fixed total marks)
4. From (2) and (1)/(3) we can derive other dependencies but the above are the main ones.

b) Candidate key(s)

A minimal key that identifies tuples in this relation is the composite:

- **Key = (RollNo, Exam_Name)**

because a student can have multiple exams and an exam has many students; the pair uniquely identifies a record of marks.

c) Check whether the schema is in 3NF

Recall: to be in **3NF** relation must be in **2NF** and have **no transitive dependencies** of a non-prime attribute on the key (i.e., every non-prime attribute must be either dependent on a key, a superkey, or be part of a candidate key).

- **Non-prime attributes** (attributes not part of any candidate key): `Branch_code`, `Marks_Obtained`, `Total_Marks`.
- `Marks_Obtained` is fully dependent on the key `(RollNo, Exam_Name)` → OK.
- **Problem (partial dependencies):**
 - `RollNo → Branch_code` — here `Branch_code` depends on **part** of the composite key `(RollNo)`, not on the whole key. This is a **partial dependency** → violates **2NF** (hence also violates 3NF).
 - `Exam_Name → Total_Marks` — `Total_Marks` depends on **part** of the composite key `(Exam_Name)` → another **partial dependency** → violates **2NF**.
- There is no transitive dependency of the form `Key → A → B` with A non-key and B non-key (except that the partial dependencies could be seen as transitive if you treat part-key → non-key), but since partial dependencies already break 2NF, the schema is **not in 3NF**.

Conclusion: The given schema is **not in 3NF**.

4. Convert the schema into 3NF (decomposition)

Decompose to eliminate partial dependencies while preserving data and enabling lossless join and dependency preservation.

Natural decomposition based on FDs:

1. Student table (store student attributes)

sql

 Copy code

```
Student(RollNo PRIMARY KEY, Branch_code)
```

FD preserved: `RollNo → Branch_code`.

2. Exam table (store exam attributes)

sql

 Copy code

```
Exam(Exam_Name PRIMARY KEY, Total_Marks)
```

FD preserved: `Exam_Name → Total_Marks`.

3. Result (or Marks) table (relationship between student and exam)

sql

 Copy code

```
Result(RollNo, Exam_Name, Marks_Obtained,  
      PRIMARY KEY (RollNo, Exam_Name),  
      FOREIGN KEY (RollNo) REFERENCES Student(RollNo),  
      FOREIGN KEY (Exam_Name) REFERENCES Exam(Exam_Name))
```

FD preserved: $(RollNo, Exam_Name) \rightarrow Marks_Obtained$.

Why this decomposition is correct

- **Removes partial dependencies:** `Branch_code` moved to `Student` (depends only on `RollNo`), `Total_Marks` moved to `Exam` (depends only on `Exam_Name`). `Marks_Obtained` remains in `Result` and depends on whole composite key.
- **Lossless-join:**
 - `Student` $\bowtie$ `Result` on `RollNo` is lossless because `RollNo` is key of `Student`.
 - `Exam` $\bowtie$ `Result` on `Exam_Name` is lossless because `Exam_Name` is key of `Exam`.Thus joining `Student`, `Exam`, and `Result` reconstructs the original relation without spurious tuples.
- **Dependency preservation:** The three FDs listed are preserved in the decomposed relations.
- **Resulting relations are in 3NF:**
 - In `Student`, key `RollNo` uniquely determines `Branch_code`. No non-key depends on non-key.
 - In `Exam`, key `Exam_Name` uniquely determines `Total_Marks`.
 - In `Result`, key $(RollNo, Exam_Name)$ uniquely determines `Marks_Obtained`.

5. Final answer (summary)

- **Functional dependencies identified:**

$RollNo \rightarrow Branch_code$

$(RollNo, Exam_Name) \rightarrow Marks_Obtained$

$Exam_Name \rightarrow Total_Marks$

- **Original schema status:** Not in 3NF (violates 2NF due to partial dependencies).
- **3NF decomposition:**

`Student(RollNo PK, Branch_code)`

`Exam(Exam_Name PK, Total_Marks)`

`Result(RollNo, Exam_Name, Marks_Obtained)` with `PK(RollNo, Exam_Name)` and FKs to `Student` and `Exam`.

10) What is decomposition? Consider the relation F (FN, PN, C, D) with the following Functional Dependencies:
FD1: FN, PN → C FD2: C → D FD3: D → F CEGP013091 49.248.216.238 16/05/2024 09:33:14 static-238 If F is decomposed into F1 (FN,PN,C) and F2 (C,D). check decomposition is lossless or lossy?

✖ 1. What is a Functional Dependency (FD)?

👉 Think of it like **cause and effect** between columns (attributes) in a table.

If one column's value **decides** another column's value,
we say that column **functionally determines** the other.

Example:

RollNo	Name	Branch
1	Riya	CS
2	Aman	IT
3	Riya	CS

Here, each **RollNo** has only one **Name** and one **Branch**.

So we can say:

- RollNo → Name
- RollNo → Branch

That's a **Functional Dependency**.

It means:

| If RollNo is same → Name & Branch will also be same.

3) Decomposition — plain meaning

- Decompose relation **R** into **R1, R2, ...** means replace a big table with smaller tables holding subsets of attributes.
- Goal: remove redundancy and anomalies while keeping the ability to reconstruct original data (by joins).

Two desired properties:

1. **Lossless-join:** join of decomposed tables returns exactly the original tuples (no spurious rows).
2. **Dependency-preserving:** all original FDs can be enforced by constraints on the new tables (without having to join).

2. What is Decomposition?

Decomposition simply means **breaking one big table into smaller tables** to remove repetition and make data cleaner.

You do this without losing any information.

Example

Let's say we have one big table called **Student_Result**:

RollNo	Name	Subject	Marks
1	Riya	Math	80
1	Riya	Science	90
2	Aman	Math	85
2	Aman	Science	88

Here, the **student name** is repeating again and again.

So we can **decompose (split)** it into two smaller tables:

1 Student Table

RollNo	Name
1	Riya
2	Aman

2 Marks Table

RollNo	Subject	Marks
1	Math	80
1	Science	90
2	Math	85
2	Science	88

Now, there's no repetition of names.

✅ We can **join both tables** (by RollNo) to get back the original big table again.

That means — **Decomposition is lossless**.

🚫 3. What is Lossless and Lossy Decomposition?

Type	Meaning	Example
Lossless	When you split a table and can join them back without losing or changing data	Splitting Student_Result into Student and Marks (we got back same data using RollNo)
Lossy	When you split and after joining back you get extra or missing rows	If we split wrongly — for example, remove RollNo and try to join only by Name — then two students with same name will mix up (wrong data)

✂️ The question in short:

We have one big table:

`R(FN, PN, C, D)`

We split (decompose) it into two smaller tables:

- `R1(FN, PN, C)`
- `R2(C, D)`

We want to check:

👉 Is this decomposition **lossless** or **lossy**?

Step 2: The rule to check

To check if decomposition is **lossless**, we look at the **common column(s)** between the two tables.

Here:

Common column = `{C}`

Now we check —

does this common column **decide all other columns** of at least one table?

That means:

👉 Does $C \rightarrow (R1 - R2)$ or $C \rightarrow (R2 - R1)$ hold true?

Step 3: What is $R1 - R2$ and $R2 - R1$?

Let's find out the attributes that are **not common**:

- $R1 - R2$ = attributes in $R1$ but **not** in $R2$
→ {FN, PN}
- $R2 - R1$ = attributes in $R2$ but **not** in $R1$
→ {D}

Step 4: Now check if common column C decides one of these.

We're given FDs:

- FD1: (FN, PN) → C
- FD2: C → D

So we can see:

✓ $C \rightarrow D$ (that's FD2)

That means **C determines D**.

And D is in $R2 - R1$.

So, $C \rightarrow (R2 - R1)$ holds true.

Step 5: What does this mean?

It means:

The common column (C) can **uniquely determine** everything in one of the decomposed tables ($R2$).

Hence, when we join $R1$ and $R2$ using C, we'll get **the same original table** — no wrong combinations or missing info.

✓ So the decomposition is **lossless**.

In short:

Step	Explanation	Result
Common column	$\{C\}$	—
Check rule	Does $C \rightarrow (R1 - R2)$ or $C \rightarrow (R2 - R1)$?	—
Find differences	$R1 - R2 = \{FN, PN\}$, $R2 - R1 = \{D\}$	—
Given FD	$C \rightarrow D$	—
So	$C \rightarrow (R2 - R1)$ holds	✓
Therefore	Decomposition is lossless	🎉

UNIT 4

11) Suppose a transaction T_i issues a read command on data item Q . How time-stamp based protocol decides whether to allow the operation to be executed or not using time-stamp based protocol of concurrency control. Explain in detail time stamp based protocol.

1 Introduction

In a multi-user database system, **Concurrency Control** is used to ensure correct results when multiple transactions execute simultaneously.

The **Timestamp Based Protocol** is a **non-locking** method of concurrency control which orders all transactions according to their **timestamps**.

2 Concept of Timestamp

- Each transaction T_i is assigned a unique **timestamp** $TS(T_i)$ when it starts.
- Timestamp indicates the **order of arrival** or **age** of the transaction.
- If $TS(T_i) < TS(T_j) \rightarrow$ Transaction T_i is older than T_j .

3 Information Stored for Each Data Item (Q)

For every data item Q , two timestamp values are maintained:

Symbol	Meaning
$read_TS(Q)$	The largest timestamp of any transaction that successfully read Q .
$write_TS(Q)$	The largest timestamp of any transaction that successfully wrote Q .

Initially, both are set to **0**.

4 Rules of Timestamp Ordering Protocol

A) When Transaction T_i issues `READ(Q)`

Let $t = TS(T_i)$

- If $t < write_TS(Q)$
 - A **younger transaction** has already written Q .
 - Reading this value will violate the timestamp order.
 - ✓ **Action: Abort T_i** and restart with new timestamp.
- Else
 - Allow the read operation.
 - Update $read_TS(Q) = \max(read_TS(Q), t)$.

B) When Transaction T_i issues `WRITE(Q)`

Let $t = TS(T_i)$

- If $t < read_TS(Q)$
 - Some younger transaction has already read Q .
 - Writing now will make older T_i overwrite a value that a newer transaction has read.
 - ✓ **Action: Abort T_i .**
- Else if $t < write_TS(Q)$
 - A younger transaction has already written Q .
 - Writing now would be out of order.
 - ✓ **Action: Either abort T_i or ignore write** (Thomas's Write Rule).
- Else
 - Perform the write.
 - Update $write_TS(Q) = t$.

5 Thomas's Write Rule (Optimization)

If a transaction tries to write an old value (its timestamp is smaller than the latest write), the write can be **ignored instead of aborting** the transaction.

👉 Example:

If $write_TS(Q)=50$ and $TS(T_i)=40$, then the write of T_i is obsolete.

Instead of aborting, just **ignore** it.

6 Example

Transaction	Operation	Condition Checked	Action	Updated Values 
T1 (TS=5)	READ(Q)	$5 \geq \text{write_TS}(Q)=0$	Allowed	$\text{read_TS}(Q)=5$
T2 (TS=10)	WRITE(Q)	$10 \geq \text{read_TS}(Q)=5$	Allowed	$\text{write_TS}(Q)=10$
T1 (TS=5)	READ(Q)	$5 < \text{write_TS}(Q)=10$	Not Allowed	T1 Aborted

✓ Hence, protocol ensures serial order = T1 before T2.

7 Advantages

1. **No Deadlocks** (no locks used).
2. Ensures **serializability** automatically.
3. **Thomas's Rule** reduces unnecessary aborts.

8 Disadvantages

1. May cause **many aborts** if older transactions are delayed.
2. Needs **unique timestamp generation**.
3. Can suffer from **starvation** of long transactions.

9 Conclusion

Timestamp based concurrency control ensures that **transactions execute in timestamp order** and maintain **serializability** without using locks.

It prevents conflicts by **aborting or ignoring** operations that violate the order.

✓ In short:

When T_i issues a read on Q , it is allowed **only if** $TS(T_i) \geq \text{write_TS}(Q)$.

Otherwise, T_i is aborted.

This maintains a conflict-free serial order based on timestamps.

12) Explain the concept of conflict serializability with suitable example. Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability?

1 Introduction

In a multi-user database, when two or more transactions execute concurrently, their operations can interleave. To ensure correctness, the result of concurrent execution must be the same as some serial execution (one after another).

This property is called **Serializability** — and it ensures consistency of the database.

There are two main types:

1. Conflict Serializability
2. View Serializability

2 Conflict Serializability – Concept

A schedule (sequence of read/write operations) is said to be conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

3 Definition of Conflicting Operations

Two operations are said to be in conflict if:

1. They belong to different transactions, and
2. They operate on the same data item, and
3. At least one of them is a WRITE operation.



4 Types of Conflicts

Operation Type	Conflict Example	Diagram
Read-Write	T1: read(Q) and T2: write(Q)	
Write-Read	T1: write(Q) and T2: read(Q)	
Write-Write	T1: write(Q) and T2: write(Q)	

5 Example of Conflict Serializability

Consider two transactions:

T1:

SCSS

 Copy code

```
read(A)
write(A)
```

T2:

SCSS

 Copy code

```
read(A)
write(A)
```

Schedule S1:

CSS

 Copy code

```
T1: read(A)
T2: read(A)
T1: write(A)
T2: write(A)
```

Let's find conflicts:

Operation Pair	Conflict Type	Order
T1.write(A) – T2.read(A)	Write–Read	T1 before T2
T1.write(A) – T2.write(A)	Write–Write	T1 before T2

👉 All conflicts show T1 → T2 ordering.

Hence, schedule is conflict equivalent to serial schedule T1 → T2,

✅ So S1 is Conflict Serializable.

6 Conflict Graph (Precedence Graph)

To test conflict serializability:

1. Create a **node** for each transaction.
2. Draw a **directed edge** $T_i \rightarrow T_j$ if an operation of T_i conflicts with an operation of T_j and T_i 's operation occurs **before** T_j 's.
3. If the graph has no **cycle**, the schedule is **conflict serializable**.

7 Example of Non-Serializable Schedule

If we get both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ edges,
→ a **cycle** occurs → **Not serializable**.

8 View Serializability

A schedule is **view serializable** if it produces the **same final results** and **reads-from relations** as some serial schedule.

It is **broader** than conflict serializability — some view-serializable schedules are not conflict-serializable.

9 Why Emphasize Conflict Serializability?

Reason	Explanation
✅ Easier to test	Conflict serializability can be checked using a simple precedence graph (acyclic test).
⚙️ Efficient	Algorithm is straightforward and computationally feasible.
🧩 View serializability is complex	Checking view serializability is NP-Complete , hence not practical for large systems.
🔗 Conflict implies view	Every conflict-serializable schedule is also view-serializable , so it's sufficient to ensure conflict serializability.

10 Conclusion

- Conflict Serializability ensures correctness by controlling the order of conflicting operations.
- It guarantees consistency and is easier to implement using precedence graph.
- Since every conflict-serializable schedule is also view-serializable,
→ DBMSs prefer conflict serializability for practical and efficient concurrency control.

✓ In short:

A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

We emphasize conflict serializability because it is easier and faster to check than view serializability.

13) State and explain the ACID properties. During its execution a transaction passes through several states, until it finally commits or aborts. List all possible sequences of states through which a transaction may pass. Explain the situations when each state transition occurs.

ACID Properties of a Transaction

Each transaction in a database must satisfy the ACID properties to ensure data reliability and consistency.

Property	Full Form	Explanation	Example
A – Atomicity	All-or-nothing	A transaction should complete fully or not at all.	If ₹500 is transferred from A → B, both debit and credit must happen; if one fails, both are undone.
C – Consistency	Data integrity maintained	Database must remain consistent before and after the transaction.	Account balances must follow rules (no negative total).
I – Isolation	Transactions independent	Transactions should not interfere with each other.	Two users withdrawing money from same account won't affect each other's balance.
D – Durability	Permanent results	Once committed, changes must survive even if system crashes.	After transfer completes, data is permanently stored in disk.

Transaction States in DBMS

A transaction moves through several states during its life cycle.

Let's explain each state in the order shown in your diagram 📌

1 Active State

- The transaction is currently executing.
- It performs **read/write** operations on data items.
- Example: While transferring money, both accounts are being updated.

→ Next:

- Goes to **Partially Committed** if all steps executed successfully.
- Goes to **Failed** if any error or crash occurs.

2 Partially Committed State

- The transaction has completed its final statement.
- Temporary changes are made in memory but not yet saved permanently.

→ Next:

- If system is fine → goes to **Committed State**.
- If crash or error → goes to **Failed State**.

3 Committed State

- All changes are **permanently saved** in the database.
- The transaction is successful and durable.

→ Next:

- Goes to **Terminated State** (finished successfully).

4 Failed State

- Transaction cannot continue due to an error, constraint violation, or system crash.

→ Next:

- System performs **Rollback** → moves to **Aborted State**.

5 Aborted State

- All temporary changes are undone (rolled back).
- Database returns to its previous consistent state.

→ Next:

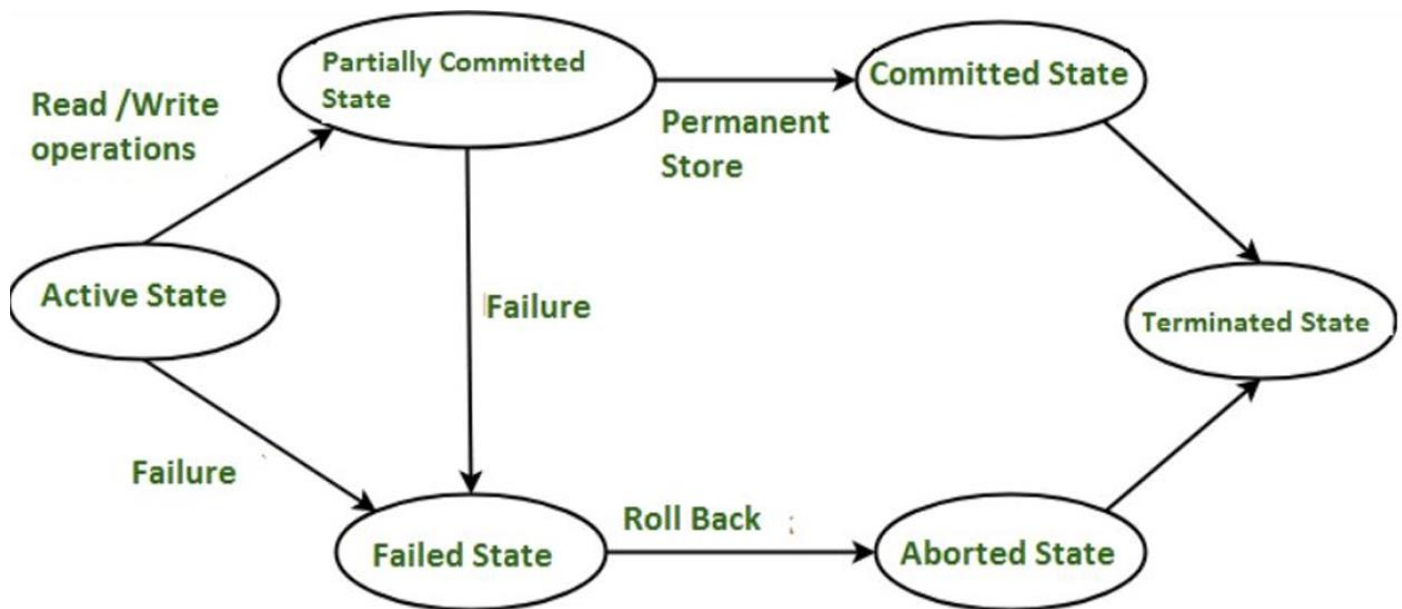
- System may restart the transaction or terminate it permanently → Terminated State.

6 Terminated State

- The transaction has completed execution — either successfully committed or aborted.
- It is now removed from the transaction table.

🔄 Possible Sequences of State Transitions

Sequence	Meaning
Active → Partially Committed → Committed → Terminated	Successful transaction ✓
Active → Failed → Aborted → Terminated	Failed transaction ✗
Active → Partially Committed → Failed → Aborted → Terminated	Failure occurred just before commit ✗



Transaction States in DBMS

14) A transaction may be waiting for more time for an Exclusive (X) lock on an item, while a sequence of other transactions request and are granted as Shared (S) lock on the same item. What is this problem? How is it solved by two phase lock protocol?

Problem Description

When a **transaction wants an Exclusive (X) lock** on a data item (to update or write it), but **other transactions keep taking Shared (S) locks** on that same item, the waiting transaction can **never get the X lock** — because S locks keep coming one after another.

 This situation is called **Starvation (or Indefinite Postponement)**

Example

Let's say we have one data item **A**, and transactions:

Transaction	Operation
T1	Wants to write(A) → needs X lock
T2	Wants to read(A) → needs S lock
T3	Wants to read(A) → needs S lock
T4	Wants to read(A) → needs S lock

What happens:

1. T1 requests an **X lock** on A — but currently, T2 already has an **S lock**.
→ So T1 has to **wait**.
2. Before T2 releases it, T3 also requests **S lock** — and gets it (since multiple S locks are allowed).
3. Then T4 requests **S lock**, and also gets it.
4. T1 keeps waiting forever for an X lock, because **there's always some S lock active**.

 This is **Starvation Problem**.

Why does it happen?

Because **X lock requests have lower priority** — every time a new transaction requests a **Shared (S) lock**, it gets it immediately, keeping the **Exclusive (X) lock request waiting** indefinitely.

Solution: Two-Phase Locking Protocol (2PL)

The **Two-Phase Locking (2PL)** protocol ensures **serializability** and helps **avoid starvation**.

◆ Two-Phase Locking Protocol Rules

Every transaction has **two phases**:

1. Growing Phase

- Transaction **can acquire locks** (S or X).
- **Cannot release** any lock.

2. Shrinking Phase

- Transaction **can release locks**,
 - **Cannot acquire** any new lock.
-

💡 How 2PL Helps

- Once a transaction releases **any lock**, it cannot obtain more locks → prevents others from entering conflicting phases indefinitely.
- By ensuring that all locks are obtained first (growing phase), transactions execute in a **conflict-serializable** manner.
- **Waiting transactions** (like one requesting an X lock) will eventually get a chance once current S locks are released — preventing starvation.

✅ Example with 2PL

1. T2, T3, T4 read A (Shared locks).
2. After finishing reads, they **release** their S locks (shrinking phase).
3. Now, T1 can **acquire the X lock** and update A (growing phase → shrinking phase).

So, the waiting transaction **eventually proceeds** — no indefinite waiting.

📄 Summary Table

Concept	Description
Problem Name	Starvation / Indefinite Postponement
Cause	Continuous granting of S locks while X lock waits
Effect	Write transaction keeps waiting forever
Solution	Two-Phase Locking Protocol (2PL)
How Solved	Ensures lock ordering, serializability, and fair execution

🚩 In short:

- When a transaction keeps waiting for X lock while others keep getting S locks → it's **Starvation**.
- **Two-Phase Locking (2PL)** protocol prevents this by controlling when locks can be acquired and released, ensuring fair and serializable execution.

15) Write a short note on : i) Log based recovery ii) Shadow Paging

15) i) Log-Based Recovery

👉 Concept:

Log-based recovery is a **technique to recover data** in a database after a **system crash or failure**. The system keeps a **log file** (or journal) that records **all operations** performed by transactions.

📄 What is a Log?

A **log** is a sequence of records that stores all the changes made by transactions. Each log record contains:

pgsql

📄 Copy code

```
<Transaction ID, Data Item, Old Value, New Value>
```

💡 Example:

If transaction T1 updates item A from 100 to 200

→ The log record will be:

```
<T1, A, 100, 200>
```

📌 Types of Log Records:

1. **<T, start>** – Transaction T has started.
2. **<T, X, old_value, new_value>** – T modified data item X.
3. **<T, commit>** – Transaction T has completed successfully.
4. **<T, abort>** – Transaction T has been rolled back.



🛠️ Recovery Process:

1. **Undo** – If a transaction **did not commit**, all its changes are **reversed** using the log's old values.
2. **Redo** – If a transaction **committed**, all its operations are **redone** using the new values in the log.

✅ Advantage:

- Ensures **Atomicity** and **Durability** in database.
- No committed data is lost even after a crash.

ii) Shadow Paging

👉 Concept:

Shadow paging is another **recovery mechanism** that avoids logging.

It uses **two copies (pages)** of data — one is the **shadow page** (original data), and the other is the **current page** (modified data).

📁 Structure:

- A **page table** stores the addresses of all data pages.
 - When a transaction starts, the system creates a **shadow copy** of this page table.
-

💡 Working:

1. The **shadow page table** always points to **stable (old)** data.
2. The **current page table** is used for modifications during the transaction.
3. If the transaction **commits**, the system **replaces the shadow table** with the current one.
4. If the transaction **fails**, the **shadow table** is kept, and changes are discarded.

🔗 Example:

Suppose a page contains value 100.

- Shadow page: 100
- Current page (after update): 200

If transaction commits → shadow updated to 200.

If abort → shadow (100) remains valid.

✅ Advantages:

- No need for undo/redo operations.
- Faster recovery after crash.

❌ Disadvantages:

- Copying large page tables is expensive.
- Not suitable for large databases.

📄 Summary Table:

Feature	Log-Based Recovery	Shadow Paging
Method	Uses log records	Uses shadow copies
Undo/Redo	Required	Not required
Overhead	Less storage overhead	High copying overhead
Suitable For	Large DB systems	Small to medium DB

✅ In short:

- **Log-based recovery** → keeps track of every change (undo/redo possible).
- **Shadow paging** → keeps two copies (safe revert possible).

